CSE 152A Introduction to Computer Vision Winter 2026 - Assignment 0

Instructor: Manmohan Chandraker

- Assignment release: **Wed, Jan 7, 2026**.
- Assignment due: **Wed, Jan 14, 2026 at 4pm PST**.

Instructions

Please answer the questions below using Python in the attached Jupyter notebook and follow the guidelines below:

- This assignment is **ungraded**, but you are highly encouraged to complete it as a test of background.
- All the solutions must be written in this Jupyter notebook.
- After finishing the assignment in the notebook, please export the notebook as a PDF and submit both the notebook and the PDF (i.e. the `.ipynb` and the `.pdf` files) on Gradescope.
- You may use basic algebra packages (such as `NumPy`, `SciPy`) to solve these problems.

Introduction

This tutorial was created by Ben Ochoa. We will use the Python programming language for assignments in this course, with a few popular libraries (`NumPy`, `Matplotlib`). Assignments will be given in the format of web-based Jupyter notebook that you are currently viewing. We expect that many of you have some experience with `Python` and `NumPy`. If you have previous knowledge in `MATLAB`, check out the NumPy for MATLAB users page. The section below will serve as a quick introduction to `NumPy` and some other libraries.

Getting Started with NumPy

`NumPy` is the fundamental package for scientific computing with Python. It provides a powerful N-dimensional array object and functions for working with these arrays. Some basic use of this packages is shown below. This is **NOT** a problem, but you are highly

recommended to run the following code with some of the input changed in order to understand the meaning of the operations.

Arrays

In [2]:
```
import numpy as np             # Import the NumPy package

v = np.array([1, 2, 3])        # A 1D array
print(v)
print(v.shape)                 # Print the size / shape of v
print("1D array:", v, "Shape:", v.shape)

v = np.array([[1], [2], [3]])  # A 2D array
print("2D array:", v, "Shape:", v.shape) # Print the size of v and check the differ

# You can also attempt to compute and print the following values and their size.

v = v.T                        # Transpose of a 2D array
m = np.zeros([3, 4])           # A 3x4 array (i.e. matrix) of zeros
v = np.ones([1, 3])            # A 1x3 array (i.e. a row vector) of ones
v = np.ones([3, 1])            # A 3x1 array (i.e. a column vector) of ones
m = np.eye(4)                  # Identity matrix
m = np.random.rand(2, 3)       # A 2x3 random matrix with values in [0, 1] (sampled
```

```
[1 2 3]
(3,)
1D array: [1 2 3] Shape: (3,)
2D array: [[1]
 [2]
 [3]] Shape: (3, 1)
```

Array Indexing

In [3]:
```
import numpy as np

print("Matrix")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m)

print("\nAccess a single element")
print(m[0, 1])                      # Access an element
m[1, 1] = 100                       # Modify an element
print("\nModify a single element")
print(m)

print("\nAccess a subarray")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m[1, :])                      # Access a row (to 1D array)
print(m[1:2, :])                    # Access a row (to 2D array)
print(m[1:3, :])                    # Access a sub-matrix
print(m[1:, :])                     # Access a sub-matrix

print("\nModify a subarray")
```

```
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
v1 = np.array([1,1,1])
m[0] = v1
print(m)
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
v1 = np.array([1,1,1])
m[:,0] = v1
print(m)
m = np.array([[1,2,3], [4,5,6], [7,8,9]]) # Create a 3x3 array.
m1 = np.array([[1,1],[1,1]])
m[:2,:2] = m1
print(m)

print("\nTranspose a subarray")
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a 3x3 array.
print(m[1, :].T)                                # Notice the difference of the dime
print(m[1:2, :].T)
print(m[1:, :].T)
print(np.transpose(m[1:, :], axes=(1,0)))       # np.transpose() can be used to tra

print("\nReverse the order of a subarray")
print(m[1, ::-1])                               # Access a row with reversed order

# Boolean array indexing
# Given a array m, create a new array with values equal to m
# if they are greater than 2, and equal to 0 if they less than or equal to 2
m = np.array([[1, 2, 3], [4, 5, 6]])
m[m > 2] = 0
print("\nBoolean array indexing: Modify with a scaler")
print(m)

# Given a array m, create a new array with values equal to those in m
# if they are greater than 0, and equal to those in n if they less than or equal 0
m = np.array([[1, 2, -3], [4, -5, 6]])
n = np.array([[1, 10, 100], [1, 10, 100]])
n[m > 0] = m[m > 0]
print("\nBoolean array indexing: Modify with another array")
print(n)
```

```
Matrix
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Access a single element
2

Modify a single element
[[  1   2   3]
 [  4 100   6]
 [  7   8   9]]

Access a subarray
[4 5 6]
[[4 5 6]]
[[4 5 6]
 [7 8 9]]
[[4 5 6]
 [7 8 9]]

Modify a subarray
[[1 1 1]
 [4 5 6]
 [7 8 9]]
[[1 2 3]
 [1 5 6]
 [1 8 9]]
[[1 1 3]
 [1 1 6]
 [7 8 9]]

Transpose a subarray
[4 5 6]
[[4]
 [5]
 [6]]
[[4 7]
 [5 8]
 [6 9]]
[[4 7]
 [5 8]
 [6 9]]

Reverse the order of a subarray
[6 5 4]

Boolean array indexing: Modify with a scaler
[[1 2 0]
 [0 0 0]]

Boolean array indexing: Modify with another array
[[  1   2 100]
 [  4  10   6]]
```

Array Dimension Operation

In [4]:
```
import numpy as np

print("Matrix")
m = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
print(m, m.shape)

print("\nReshape")
re_m = m.reshape(1,2,2)  # Add one more dimension at first.
print(re_m, re_m.shape)
re_m = m.reshape(2,1,2)  # Add one more dimension in middle.
print(re_m, re_m.shape)
re_m = m.reshape(2,2,1)  # Add one more dimension at last.
print(re_m, re_m.shape)

print("\nStack")
m1 = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
m2 = np.array([[1, 1], [1, 1]]) # Create a 2x2 array.
print(np.stack((m1,m2)))

print("\nConcatenate")
m1 = np.array([[1, 2], [3, 4]]) # Create a 2x2 array.
m2 = np.array([[1, 1], [1, 1]]) # Create a 2x2 array.
print(np.concatenate((m1,m2)))
print(np.concatenate((m1,m2), axis=0))
print(np.concatenate((m1,m2), axis=1))
```

```
Matrix
[[1 2]
 [3 4]] (2, 2)

Reshape
[[[1 2]
  [3 4]]] (1, 2, 2)
[[[1 2]]

 [[3 4]]] (2, 1, 2)
[[[1]
  [2]]

 [[3]
  [4]]] (2, 2, 1)

Stack
[[[1 2]
  [3 4]]

 [[1 1]
  [1 1]]]

Concatenate
[[1 2]
 [3 4]
 [1 1]
 [1 1]]
[[1 2]
 [3 4]
 [1 1]
 [1 1]]
[[1 2 1 1]
 [3 4 1 1]]
```

Math Operations on Array

Element-wise Operations

In [5]:
```python
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
print(a * 3)                                          # Scalar multiplication
print(a / 2)                                          # Scalar division
print(np.round(a / 2))
print(np.power(a, 2))
print(np.log(a))
print(np.exp(a))

b = np.array([[1, 1, 1], [2, 2, 2]], dtype=np.float64)
print(a + b)                                          # Elementwise sum
print(a - b)                                          # Elementwise difference
print(a * b)                                          # Elementwise product
print(a / b)                                          # Elementwise division
print(a == b)                                         # Elementwise comparison
```

```
[[ 3.  6.  9.]
 [12. 15. 18.]]
[[0.5 1.  1.5]
 [2.  2.5 3. ]]
[[0. 1. 2.]
 [2. 2. 3.]]
[[ 1.  4.  9.]
 [16. 25. 36.]]
[[0.         0.69314718 1.09861229]
 [1.38629436 1.60943791 1.79175947]]
[[  2.71828183   7.3890561   20.08553692]
 [ 54.59815003 148.4131591  403.42879349]]
[[2. 3. 4.]
 [6. 7. 8.]]
[[0. 1. 2.]
 [2. 3. 4.]]
[[ 1.  2.  3.]
 [ 8. 10. 12.]]
[[1.  2.  3. ]
 [2.  2.5 3. ]]
[[ True False False]
 [False False False]]
```

Broadcasting

In [6]:
```
# Note: See https://numpy.org/doc/stable/user/basics.broadcasting.html
#       for more details.
import numpy as np
a = np.array([[1, 1, 1], [2, 2, 2]], dtype=np.float64)
b = np.array([1, 2, 3])
print(a*b)
```

```
[[1. 2. 3.]
 [2. 4. 6.]]
```

Sum and Mean

In [7]:
```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("Sum of array")
print(np.sum(a))                # Sum of all array elements
print(np.sum(a, axis=0))        # Sum of each column
print(np.sum(a, axis=1))        # Sum of each row
print("\nMean of array")
print(np.mean(a))               # Mean of all array elements
print(np.mean(a, axis=0))       # Mean of each column
print(np.mean(a, axis=1))       # Mean of each row
```

```
Sum of array
21
[5 7 9]
[ 6 15]

Mean of array
3.5
[2.5 3.5 4.5]
[2. 5.]
```

Vector and Matrix Operations

In [8]:
```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[1, 1], [1, 1]])
print("Matrix-matrix product")
print(a.dot(b))                 # Matrix-matrix product
print(a.T.dot(b.T))

x = np.array([3, 4])
print("\nMatrix-vector product")
print(a.dot(x))                 # Matrix-vector product

x = np.array([1, 2])
y = np.array([3, 4])
print("\nVector-vector product")
print(x.dot(y))                 # Vector-vector product
```

```
Matrix-matrix product
[[3 3]
 [7 7]]
[[4 4]
 [6 6]]

Matrix-vector product
[11 25]

Vector-vector product
11
```

Matplotlib

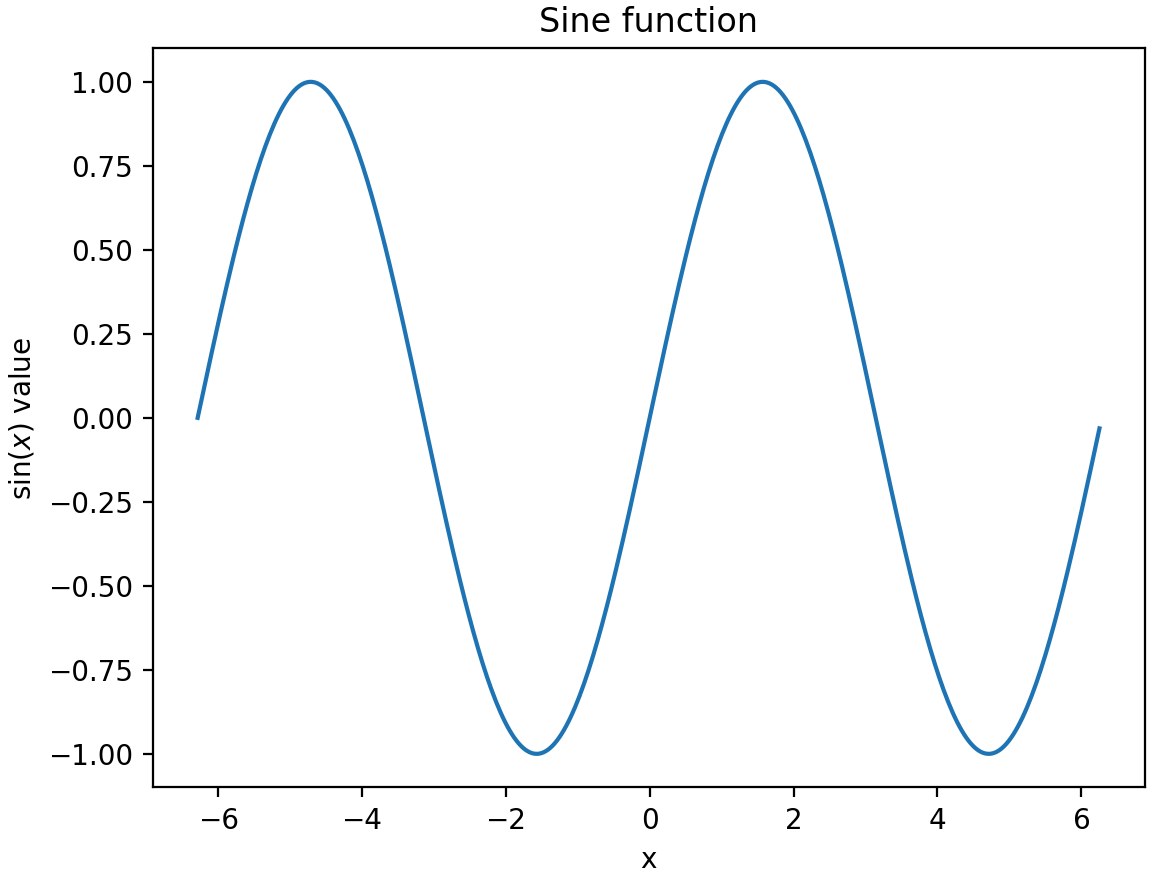
`Matplotlib` is a plotting library. We will use it to show the result in this assignment.

In [9]:
```
%config InlineBackend.figure_format = 'retina' # For high-resolution.
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-2., 2., 0.01) * np.pi
plt.plot(x, np.sin(x))
plt.xlabel('x')
plt.ylabel('$\sin(x)$ value') # '$...$' for a LaTeX formula.
plt.title('Sine function')
```

```
plt.show()
```

```
<>:8: SyntaxWarning: invalid escape sequence '\s'
<>:8: SyntaxWarning: invalid escape sequence '\s'
C:\Users\issac\AppData\Local\Temp\ipykernel_16044\1523278240.py:8: SyntaxWarning: in
valid escape sequence '\s'
  plt.ylabel('$\sin(x)$ value') # '$...$' for a LaTeX formula.
```



This brief overview introduces many basic functions from `NumPy` and `Matplotlib`, but is far from complete. Check out more operations and their use in documentations for NumPy and Matplotlib.

Problem 1: Image Operations and Vectorization (5 points)

Vector operations using `NumPy` can offer a significant speedup over doing an operation iteratively on an image. The problem below will demonstrate the time it takes for both approaches to change the color of quadrants of an image.

The problem reads an image `ucsd-triton-statue.png` that you will find in the assignment folder. Two functions are then provided as different approaches for doing an operation on the image.

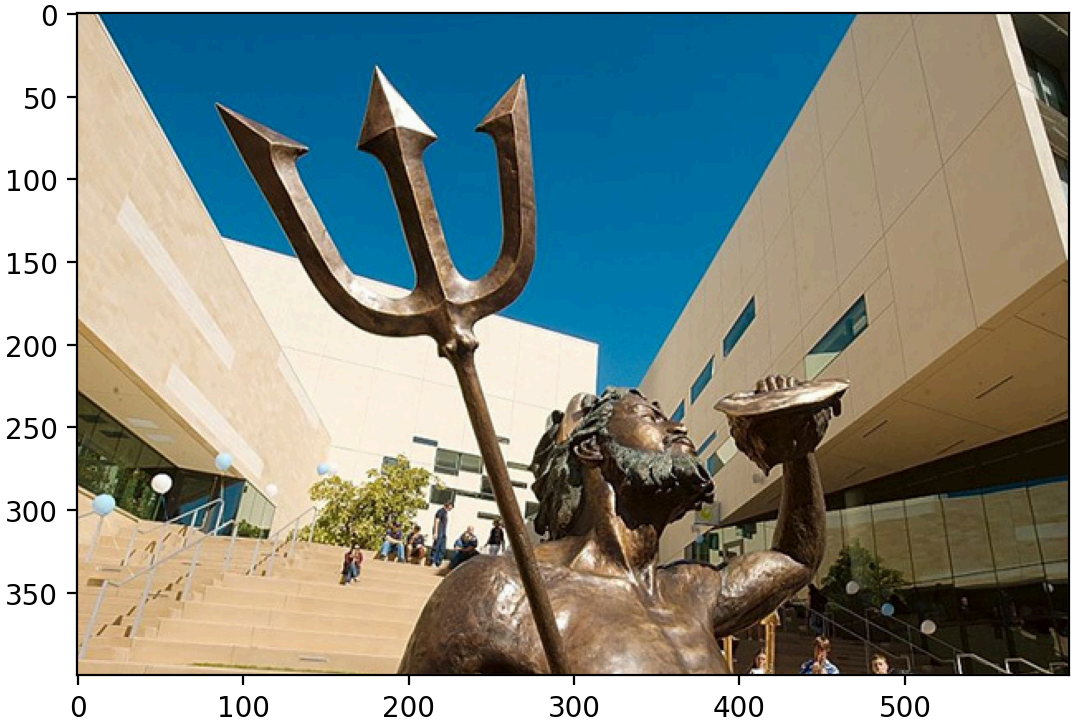
Your task is to follow through the code and fill the blanks in `vectorized()` function and compare the speed difference between `iterative()` and `vectorized()`.

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
import copy
import time

img = plt.imread('ucsd-triton-statue.jpg') # Read an image
print("Image shape:", img.shape)           # Print image size and color depth. The

plt.imshow(img)                            # Show the original image
plt.show()
```

```
Image shape: (400, 600, 3)
```



In [11]:
```
def iterative(img):
    """ Iterative operation. """
    image = copy.deepcopy(img)              # Create a copy of the image matrix
    for x in range(image.shape[0]):
        for y in range(image.shape[1]):
            if x < image.shape[0]/2 and y < image.shape[1]/2:
                image[x,y] = image[x,y] * np.array([1,0,0])    # Keep the red chann
            elif x > image.shape[0]/2 and y < image.shape[1]/2:
                image[x,y] = image[x,y] * np.array([0,1,0])    # Keep the green cha
            elif x < image.shape[0]/2 and y > image.shape[1]/2:
                image[x,y] = image[x,y] * np.array([0,0,1])    # Keep the blue chan
            else:
                pass
    return image

def vectorized(img):
    """ Vectorized operation. """
```

```
    image = copy.deepcopy(img)
    a = int(image.shape[0]/2)
    b = int(image.shape[1]/2)
    image[:a,:b] = image[:a,:b]*np.array([1,0,0])   # Keep the red channel

    # Please also keep the green channel / blue channel respectively in image[a:, :
    # with the vectorized operation as shown above. You need to make sure your fina
    # vectorized() function is the same as the one generated from iterative().

    #### Write your code here. ####
    image[a:,:b] = image[a:, :b] * np.array([0, 1, 0]) # Keep the green channel

    image[:a,b:] = image[:a, b:] * np.array([0, 0, 1]) # Keep the blue channel

    return image
```

Now, run the following cell to compare the difference between iterative and vectorized operation.

In [12]:
```
import time

def compare():
    img = plt.imread('ucsd-triton-statue.jpg')
    cur_time = time.time()
    image_iterative = iterative(img)
    print("Iterative operation (sec):", time.time() - cur_time)

    cur_time = time.time()
    image_vectorized = vectorized(img)
    print("Vectorized operation (sec):", time.time() - cur_time)

    return image_iterative, image_vectorized

# Run the function
image_iterative, image_vectorized = compare()

# Plotting the results in sepearate subplots.
plt.figure(figsize=(12,4))   # Adjust the figure size.
plt.subplot(1, 3, 1)         # Create 1x3 subplots, indexing from 1
plt.imshow(img)              # Original image.

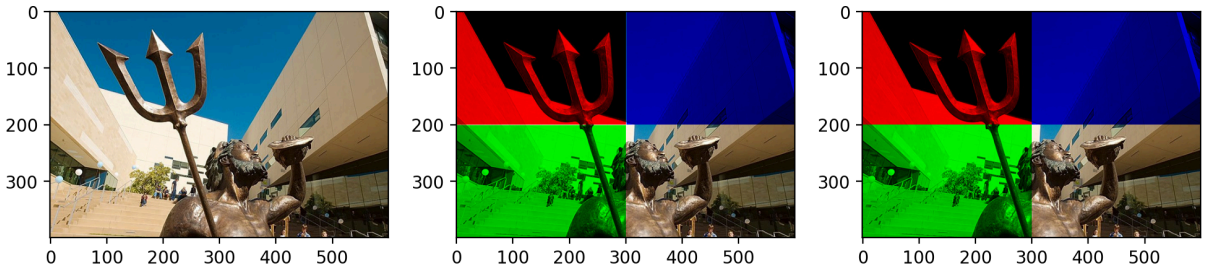
plt.subplot(1, 3, 2)
plt.imshow(image_iterative)  # Iterative operations on the image.

plt.subplot(1, 3, 3)
plt.imshow(image_vectorized) # Vectorized operations on the image.

plt.show()                   # Show the figure.

# Note: The shown figures of image_iterative and image_vectorized should be identic
```

```
Iterative operation (sec): 0.7328073978424072
Vectorized operation (sec): 0.0050160884857177734
```



Problem 2: More Image Operations (45 points)

In this problem you will reuse the image `ucsd-triton-statue.png`. Being a color image, this image has three channels, corresponding to the primary colors of red, green and blue.

(1) Read the image.

(2) Write your implementation to extract each of these channels separately to create single channel images. This means that from the $H \times W \times 3$ shaped image, you'll get three matrices of the shape $H \times W$ (Note that it's 2-dimensional).

(3) Now, write a function to merge all these single channel images back into a 3-dimensional colored image. Merge the 2D images using the original channels order (R,G,B) and the reversed channels order (B,G,R).

(4) Next, write another function to mirror the original image from left to right. For this function, please only use **array indexing** to implement this function and **do not** directly use NumPy functions (such as `np.flip()`) that flip the matrix.

(5) Next, write another function to rotate the original image 90 degrees counterclockwise. For this function, please only use **array indexing** to implement this function and **do not** directly use NumPy functions (such as `np.rot90()`) that directly rotate the matrix. Try to apply the rotation function once (that is, a 90-degree rotation) and twice (that is, a 180-degree rotation).

(6) Finally, consider **4 color images** you obtained: 2 from merging (RGB and BGR), 1 from mirroring (left to right) and 1 from rotation (180-degree). Using these 4 images, create one single image by tiling them together **without using loops**. The image will have 2×2 tiles making the shape of the final image $2H \times 2W \times 3$. The order in which the images are tiled does not matter. Show the tiled image.

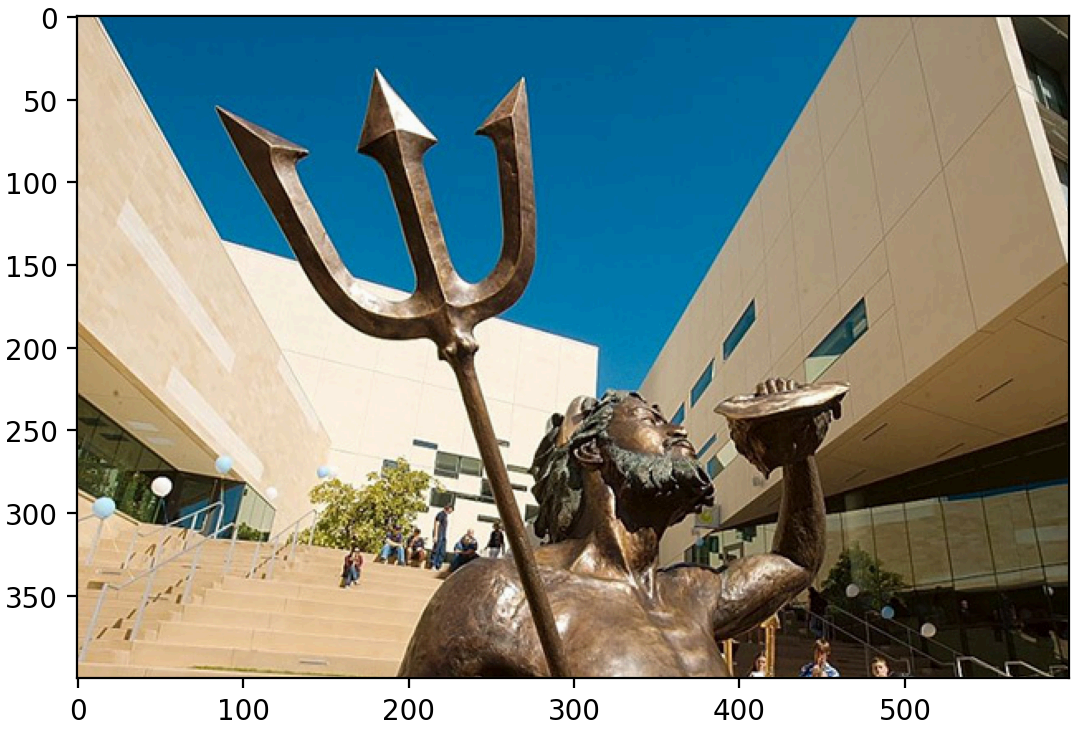
In [13]:
```
import numpy as np
import matplotlib.pyplot as plt
import copy
```

In [14]:
```
# (1) Read the image.
#### Write your code here. ####
img = plt.imread('ucsd-triton-statue.jpg')
```

```
plt.imshow(img) # Show the image after reading.
plt.show()
```

```
# (2) Extract single channel image.
def get_channel(img, channel):
    """ Function to extract 2D image corresponding to a channel index from a color
    This function should return a H*W array which is the corresponding channel of t
    #### Write your code here. ####
    return img[:, :, channel]

# Test your implemented get_channel()
assert len(get_channel(img, 0).shape) == 2  # Index 0
```



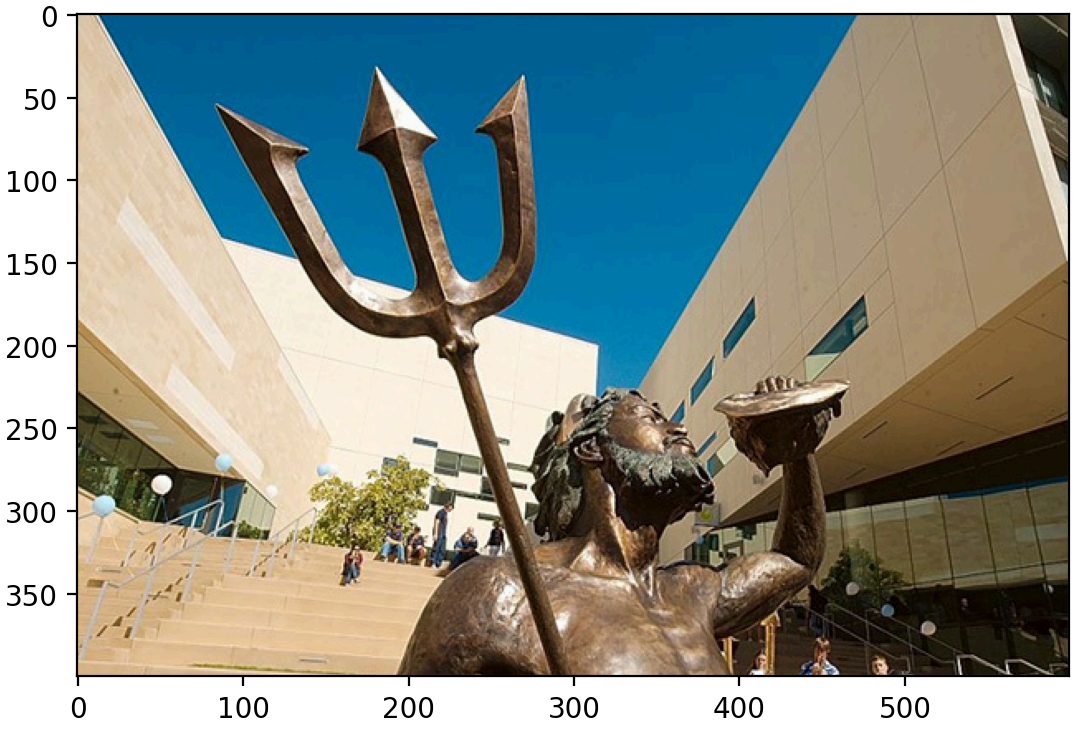
In [15]:
```
# (3) Merge channels.
def merge_channels(img0, img1, img2):
    """ Function to merge three single channel images to form a color image.
    This function should return a H*W*3 array which merges all three single channel
    (i.e. img0, img1, img2) in the input."""
    # Hint: There are multiple ways to implement it.
    #       1. For example, create a H*W*C array with all values as zero and
    #          fill each channel with given single channel image.
    #          You may refer to the "Modify a subarray" section in the brief NumPy
    #       2. You may find np.stack() / np.concatenate() / np.reshape() useful in

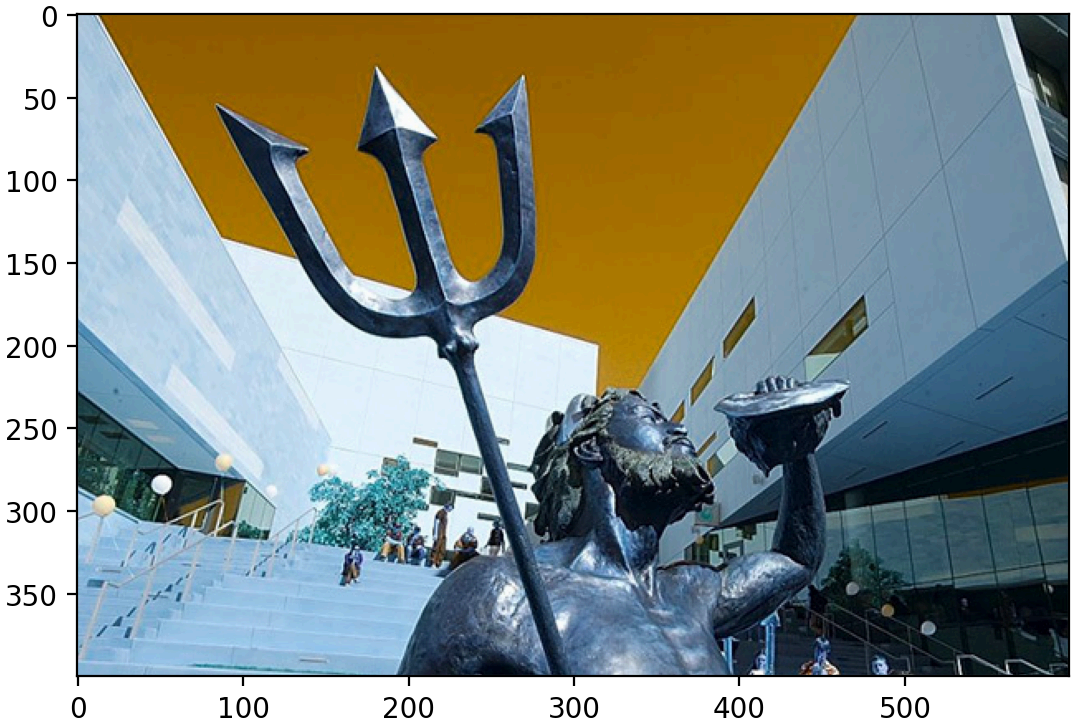
    #### Write your code here. ####
    return np.stack([img0, img1, img2], axis=2)

img0 = get_channel(img, 0)  # Get single channel images.
```

```
img1 = get_channel(img, 1)
img2 = get_channel(img, 2)

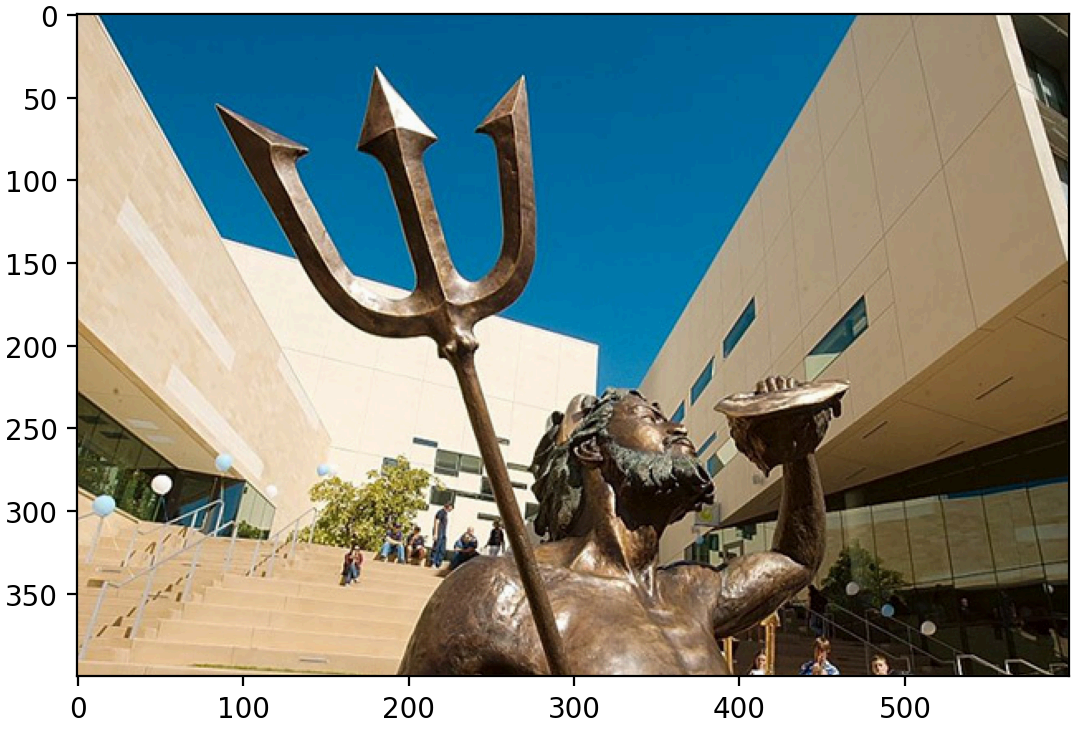
#### Write your code here. ####
RGB_img = merge_channels(img0, img1, img2) # Merge the channels in R,G,B order (the
BGR_img = merge_channels(img2, img1, img0) # Merge the channels in B,G,R order (swa
plt.imshow(RGB_img)
plt.show()
plt.imshow(BGR_img)
plt.show()
```

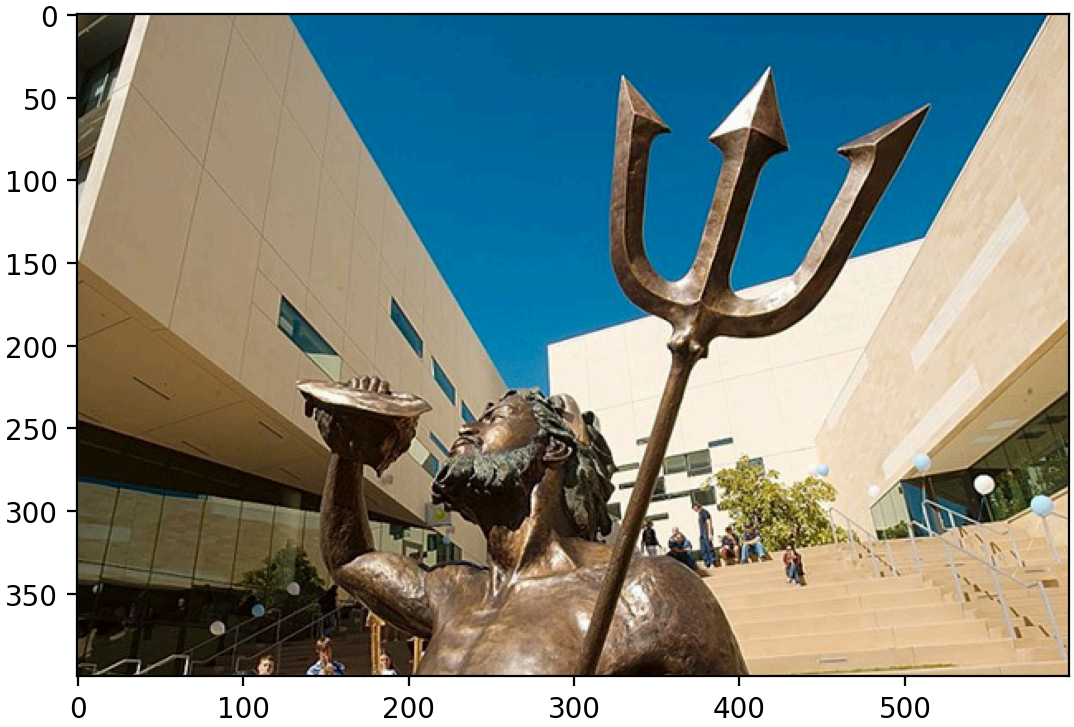




In [16]:
```
# (4) Mirror the image from left to right.
def mirror_img(img):
    """ Function to mirror image from left to right.
    This function should return a H*W*3 array which is the mirrored version of orig
    """
    #### Write your code here. ####
    return img[:, ::-1, :]

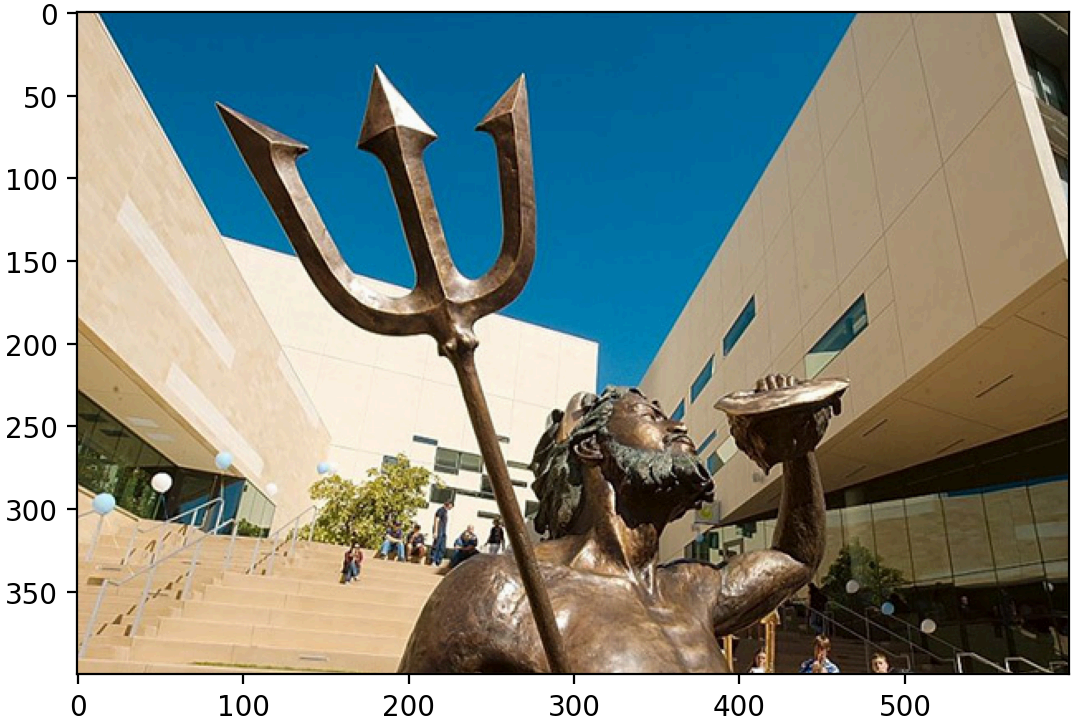
plt.imshow(img)
plt.show()
mirrored_img = mirror_img(img)
plt.imshow(mirrored_img)
plt.show()
```

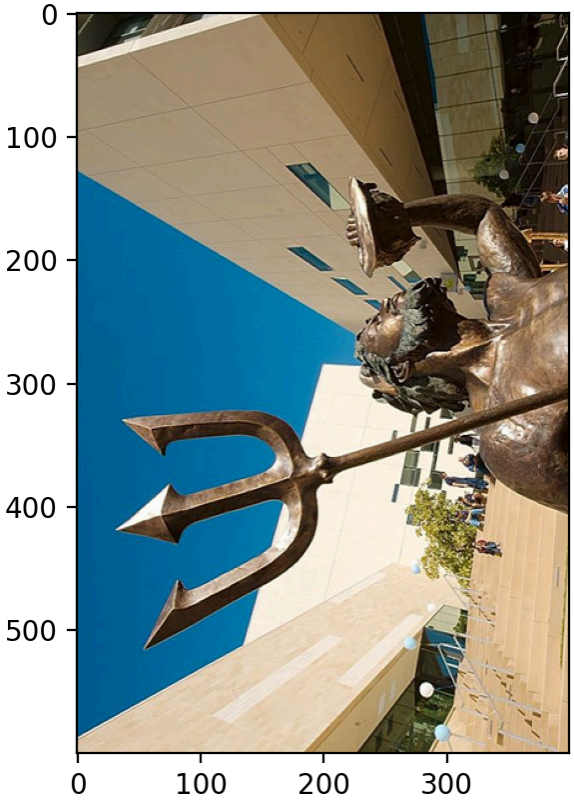


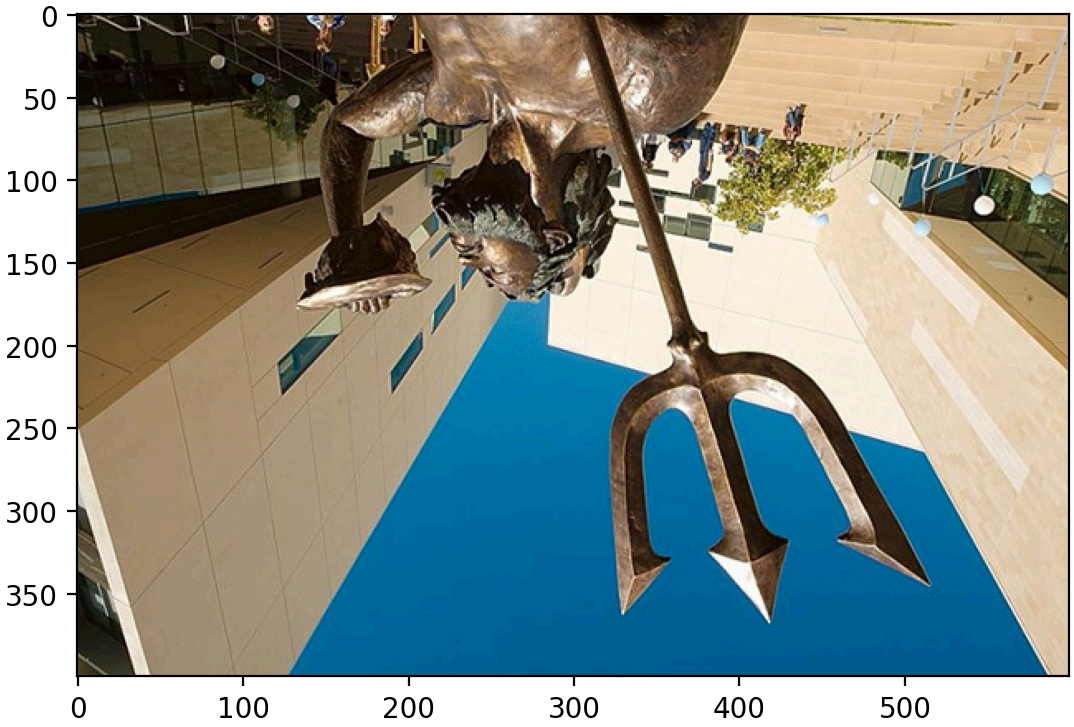


In [17]:
```python
# (5) Rotate image.
def rotate_img(img):
    """ Function to rotate image 90 degrees counter-clockwise.
    This function should return a W*H*3 array which is the rotated version of origi
    #### Write your code here. ####
    return img.transpose(1, 0, 2)[::-1]

plt.imshow(img)
plt.show()
rot90_img = rotate_img(img)
plt.imshow(rot90_img)
plt.show()
rot180_img = rotate_img(rotate_img(img))
plt.imshow(rot180_img)
plt.show()
```

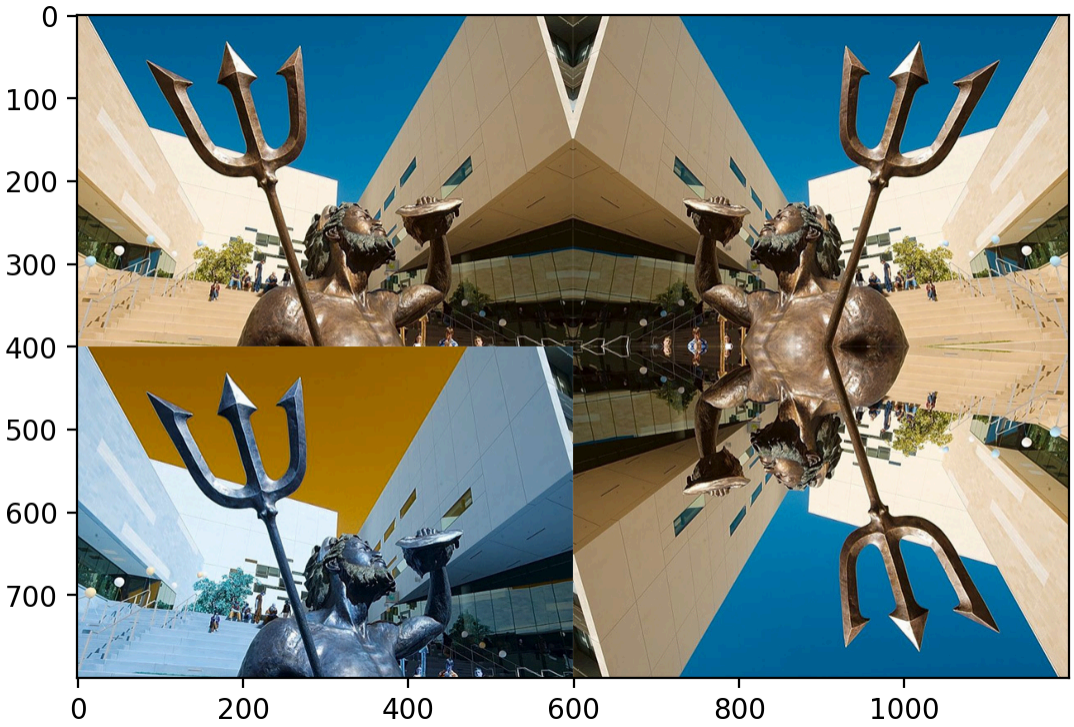

0
50
100
150
200
250
300
350
0
100
200
300
400
500


0
100
200
300
400
500
0
100
200
300



In [18]:
```
# (6) Write your code here to tile the four images and make a single image.
# You can use the RGB_img, BGR_img, mirrored_img, rot180_img to represent the four
# After tiling, please display the tiled image.

#### Write your code here. ####
images = np.concat([np.concat([RGB_img, BGR_img]), np.concat([mirrored_img, rot180_
plt.imshow(images)
plt.show()
```



Submission Instructions

Remember to submit **both** the Jupyter notebook file (.ipynb) and the PDF version of this notebook to Gradescope. Please make sure the content in each cell is clearly shown in your final PDF file. To convert the notebook to PDF, you can choose one way below:

The recommended thing to ensure cells and output are not cutoff in the PDF is to:

1. Export the notebook as HTML. (File -> Save and export or Export)
2. Open the HTML in the browser of your choice.
3. Right click on the webpage and click print.
4. Make sure the destination is "Save as PDF".